Using Babelsberg/RML – An Executable Formal Semantics and Language Testing Tool

Tim Felgentreff, Todd Millstein, Alan Borning, and Robert Hirschfeld

August 7, 2015

1 Getting Started

This is an artifact to go with the paper "Checks and Balances - Constraint Solving without Surprises in Object-Constraint Programming Languages". It consists of this file and an `OVA` VM package. `OVA` is an open format to package virtual machines and is supported by virtualization software such as VMware Player or Oracle Virtual Box. This artifact was created and tested on Virtual Box. To get started, import the `OVA` into Virtual Box by clicking "File→Import Appliance" and following the wizard. Once the VM is set up, start it. It will log in automatically, start a graphical window manager, and open a terminal.

You are logged in as user `oopsla`. (The user has sudo access should you need it; the password is `artifact`.)

The home directory includes the following 4 folders:

- `babelsberg-rml`—the executable semantics
- `babelsberg-js`—the JavaScript implementation of Babelsberg, including the changes required to make it more conformant with the semantics.
- `babelsberg-r`—the Ruby implementation of Babelsberg
- `babelsberg-s`—the Squeak/Smalltalk implementation of Babelsberg

To test that everything is working as it should, execute the following commands to enter the `babelsberg-rml` folder and run the basic test suite:

```
cd babelsberg-rml
rake objects:citest
```

This will run the tests that can be found in the companion technical report to the OOPSLA paper, and for each invocation of the solver it will print the translated constraints and the resulting environment. If all is well, this should exit with an exitcode of 0 (the prompt will show a green colon, not a red exclamation mark). The last line printed should read `All OK!`.

2 Step-By-Step Instructions

The VM includes a rudimentary graphical environment, the Chromium web browser, and the Emacs, VIM, and Nano editors. It does not require internet access; the web browser is only used to access a local server for Babelsberg/JS.

2.1 Babelsberg/RML Executable Semantics

Folder Layout The `babelsberg-rml` folder contains the implementations of the different executable semantics for Babelsberg in the subfolders `reals`, `records`, `uid`, and `objects`. The `uid` folder implements the semantics as presented in the OOPSLA paper, the `reals` and `records` folders contain semantics with fewer features (only primitives and primitives+immutable records, respectively). The `objects` folder contains the semantics for the full object-oriented language with methods. All of these are described in detail in the companion technical report to the OOPSLA paper.

The interesting file to look at in each of these subdirectories is `babelsberg.rml`. It contains the definition of the abstract syntax and the rules as presented in the paper and companion technical report. The relations are named after the judgments in the formal semantics, and the rules syntax closely matches the formal rules, with preconditions written first and the transformation rule given below the `----` line. For an introduction to RML, please see [1].

Each semantics subfolder furthermore includes an `examples` folder which contains the programs that are presented (file ending `txt`) in the technical report, the expected solutions to each invocation of the solver for each program (file ending `env`), and possibly a marker file that declares that a program is invalid (file ending `illegal`). Furthermore, a small number of examples also come with a file ending in `z3rb`. This is to workaround what we think are bugs in the version of Z3 that we use that produce wrong results depending on the order of type declarations.

Finally, the `babelsberg-rml` folder contains, besides the different semantics, the folders `javascript`, `ruby`, and `squeak`. These contain the framework (also implemented in RML) for transforming the tests in `spec/examples` into tests for each language implementation. The language specific transformations are defined in the language-specific RML files, and a language specific scaffold file is used to setup the test environment. Each folder already includes a generated testsuite file.

Working with Babelsberg/RML Babelsberg/RML is managed with `rake`. Execute the following commands to see the available tasks:

```
cd && cd babelsberg-rml
rake -T
```

Most tasks are available for each of the semantics. They are invoked using `rake specname:taskname`. Here is a listing of all the task types, followed by a step-by-step example of running one of these tasks.

- `specname:build`—compile the semantics `specname`. There should be no need to run this manually, as it is run automatically as needed.
- `specname:program`—this will execute the semantics on a program specified in the `specname/program.txt` file. This task is useful to quickly try different programs.
- `specname:review`—This will execute one or all examples in the `specname/examples` directory. The optional parameter is given in brackets directly after the invocation (e.g. `rake uid:review[12]`). This task

will run the example and stop at each invocation of the solver to show the current set of constraints, and prompt the user to enter a valid solution to those constraints. This solution is then translated back into the updated environment (and heap, for the `uid` and `objects` semantics) and the program continues executing.

- `specname:test`—Like the previous task, this task will run the specified example and stop at each invocation of the solver to show the current set of constraints, but it will also show the solution that was specified as expected in the semantics (given in the `env` file for the example). The user can then review the constraints and solution to check for errors in the constraint generation or mistakes in the solution.

- `specname:citest`—This task will run the specified example, but instead of stopping it will just use the solutions provided in the `env` file, thus automatically providing the solutions. If the solutions are valid, the program should run to completion (or fail, if it was marked as failing). This tasks should be successful for all semantics and examples.

Only for Babelsberg/Objects do we provide tasks which run the examples against the Z3 constraint solver. (Z3 supports records, booleans, reals, and integers natively, and we represent strings as decimal-coded characters.)

- `objects:z3`—This task will run the example and pass the constraints to the Z3 constraint solver. The solution can then be reviewed before continuing.

- `objects:z3ci`—This task will run the example and pass the constraints to the Z3 constraint solver. Each solution is checked against the `env` file for the example (as taken from the technical report) and the task aborts if the solutions do not agree. This task was mainly used to check the examples in the paper.

- `objects:z3run`—This task will run an example and use Z3 for solving, but will not compare the solution against the expected solution.

Here is a step-by-step description of how to run the `reals:program` task. Hopefully it will be clear from this how to run the others. For this task, as well as the other tasks without "z3" in the task name, the user needs to play the role of the constraint solver. To support this, whenever a constraint solution is needed, the system will pop up a text editor that asks the user to enter the correct solution to the constraints. For the `objects:z3` tasks, the system will give the constraints to Z3 to solve, but will pop up a text editor to ask whether the solution Z3 returns is correct.

The default is that the editor is Nano; to change this to another editor, set the environment variable `$BBBEDITOR`. For example, to change to Emacs add the line `EXPORT BBBEDITOR="emacs"` to the file `.bashrc` in the `/home/oopsla` directory and then say `source .bashrc` to read it in.

In the directory `/home/oopsla/babelsberg-rml`, type `rake reals:program`. This will execute the semantics on the file `/home/oopsla/babelsberg-rml/reals/program.txt`, prompting the user along the way to solve the constraints. The contents of this file are as follows:

```
a := 1;
always required a = 2;
a := 5
```

(Note that due to a simplifcation in the parser, the last line *must not* have a semi-colon).

Right after typing `rake reals:program`, a text editor should pop up, asking you to solve the first set of constraints:

```
required (a = 1.0)
```

(There will also be some `required true` constraints, which we omit.)

In the text editor, type the solution:

```
a 1.0
```

Then save and exit. Next you'll see a new set of constraints to solve:

```
weak (a = 1.0) &&
required (a = 2.0)
```

These arise after adding the `required a=2` constraint. Type in the solution `a 2`. Finally, you'll be asked to solve

```
required (a = 2.0) &&
weak (a = 2.0) &&
required (a = 5.0)
```

These are unsolvable, so just type in `unsat`.

The system will respond `Evaluation failed!`, which is what we want — the required constraint `a=2` and the other required constraint `a=5`, which arises from the last assignment, are incompatible.

You (as the constraint solver) are presumed to be sound, but of course you can lie ... for example, if rather than `unsat` for the last problem, you type in `a 42`, that is the answer that the system will return.

Writing own examples The easiest way to try own examples is to edit the `program.txt` file in the folder for which you want to try the example, and then use `rake specname:program` to run it. Take a look into the examples folder for the syntax. Operators have precedence as in mathematical notation, each statement must end with a semi-colon *except* the last, which must not have a semi-colon. Syntax-errors are, unfortunately, not very useful. If you do run into syntax errors, try to compare your program with the provided examples, as well as the syntax definition given in the `babelsberg.rml` files.

How to Generate Tests To generate one test or the entire test suite for the languages, the tasks `javascript`, `ruby`, and `squeak` can be used. Without an argument, these transform the entire suite contained in `babelsberg-rml/spec/examples` into the language specific code; otherwise just a single example is transformed. By default, the output is printed to the terminal. Redirect to the language specific file to run it. For example, to generate `test2` for Javascript, based on the tests in `babelsberg-rml/spec/examples/test2.txt`, and save it in a file `mytest2.js`,

give this command:
`rake javascript[2] > mytest2.js`
See Section 2.2 for how to then run `mytest2.js`.

Details Regarding Problem Tests for `objects:z3ci` Here are details regarding the `z3rb` files that come with some tasks: Due to a bug in the current version of the Z3 optimizer (at least *we* think it's a bug), the Z3 model finder will pick an empty record when it should pick a 0 for a variable that has only weak constraints on it. If we change the datatype declaration in the `babelsberg.rml` files, which reads

```
(declare-datatypes ()
       ((Value (Record (rec (Array Label Real)))
               (Real (real Real))
               (Bool (bool Bool))
               (Reference (ref Reference)))))
```

to instead say

```
(declare-datatypes ()
       ((Value (Real (real Real))
               (Record (rec (Array Label Real)))
               (Bool (bool Bool))
               (Reference (ref Reference)))))
```

The `z3rb` simply do string-manipulation on the constraints that are generated from the executable semantics before they are passed to Z3. Another use we make for this is to encode the string concatenation operation in example 10, which Z3 does not natively support, so that it returns the string encoding that we have chosen.

2.2 Language Test Suites

The VM provides global commands to start each of the implemented languages. The repositories are only provided for those interested in the implementation.

Babelsberg/JS

The `babelsberg-js` folder contains the JavaScript implementation of Babelsberg. It can run in the Lively Kernel and as a standalone JavaScript project. To run tests generated from `babelsberg-rml` (through e.g. `rake javascript > testsuite.js`, copy the generated JS file into `$HOME/babelsberg-js/babelsberg/testsuite.js`. Then run the following command:

```
babelsbergjs
```

This will run the Lively Kernel server, a copy of which is included with this artifact, and start the Chromium web browser to view a Lively Kernel page with a test runner. The testsuite is already loaded and can be run directly. This is the modified version of Babelsberg/JS that passes most tests. As with the RML semantics, it cannot work with strings, which is why tests 9, 10, and 11 fail. Tests 15, 20, and 32 test properties of value classes. As we have described

in the paper, these do not work in Javascript. The remaining two failures, tests 4 and 32b, simply produce other, but equally valid solutions.

Babelsberg/R

The `babelsberg-r` directory contains the repository for Babelsberg/R, based on the Topaz Ruby VM. A precompiled binary is provided. The test suite can be run using the command:

```
babelsbergr $HOME/babelsberg-rml/ruby/testsuite.rb
```

Babelsberg/S

The `babelsberg-s` directory contains the repository for Babelsberg/S, and a prepared Squeak Image. To start it, just run:

```
babelsbergs
```

In the image, you can run the test suite using the open Test Runner, by scrolling the first pane to the very bottom and selecting "Babelsberg-Tests", and then in the second pane selecting "BabelsbergTestSuite". To load a new version of the test suite, use the "File List" window that is open in the lower left corner. It has the generated file already selected. Click the "filein" button to load the tests and re-execute them.

References

[1] Peter Fritzson. Developing efficient language implementations from structural and natural semantics. Retrieved on 08-Jun-2015: `http://www.ida.liu.se/~{}pelab/rml/rml-ftp/papers/rmlguide-v22-pdf-060314.pdf`.